

Ayudantía 8: bases de datos relacionales

De archivos a modelos relacionales y consultas SQL

IIC2115

Objetivo del repaso

Idea central

Pasar de datos en archivos a una base relacional bien modelada y lista para consultas.

- ▶ Entender entidades y relaciones.
- ▶ Definir llaves primarias y foráneas.
- ▶ Crear tablas con SQL desde Python.
- ▶ Insertar datos desde CSV/JSON.
- ▶ Consultar datos con SQL.
- ▶ Entender JOIN, GROUP BY, HAVING.
- ▶ Evitar errores típicos de SQLite.
- ▶ Saber cómo enfrentar ejercicios nuevos.

Lo que más apareció en el ticket de salida

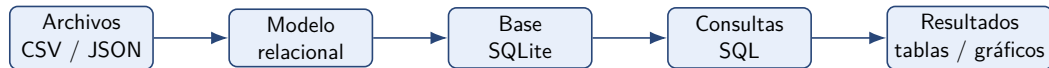
Dudas conceptuales

- ▶ ¿Cómo identificar entidades?
- ▶ ¿Cuándo separar información en otra tabla?
- ▶ ¿Cómo definir cardinalidad?
- ▶ ¿Qué son realmente PK y FK?
- ▶ Diferencia entre DataFrame y base de datos.

Dudas técnicas

- ▶ Crear tablas desde Python.
- ▶ Insertar datos desde CSV/JSON.
- ▶ Ver dónde queda creada la base.
- ▶ Errores al correr dos veces.
- ▶ Consultas SQL, especialmente JOIN.

El pipeline mental



Error común

Intentar programar antes de entender el modelo. SQL no parte en el código, sino decidiendo qué tablas existen y cómo se conectan.

DataFrame vs base de datos

DataFrame	Base de datos relacional
Objeto en memoria de Python. Sirve para explorar y transformar datos.	Archivo o sistema persistente. Sirve para guardar, ordenar y consultar datos.
Normalmente una tabla aislada. Relaciones no se protegen solas.	Varias tablas relacionadas entre sí. Puede usar restricciones: PK, FK, UNIQUE.
Se manipula con pandas.	Se manipula con SQL.

Resumen

Un DataFrame puede ser el punto de partida, pero la base relacional exige decidir estructura, llaves y relaciones.

¿Qué es una entidad?

Definición práctica

Una entidad es una “cosa” del problema sobre la cual queremos guardar información propia.

Ejemplo: vehículos

- ▶ Vehículo
- ▶ Marca
- ▶ Modelo
- ▶ Dueño
- ▶ Revisión técnica

Preguntas guía

- ▶ ¿Tiene atributos propios?
- ▶ ¿Se repite muchas veces?
- ▶ ¿Puede relacionarse con varias filas?
- ▶ ¿Conviene evitar duplicación?

¿Cuándo separar en otra tabla?

Conviene separar si...

- ▶ La información se repite mucho.
- ▶ Tiene atributos propios.
- ▶ Puede existir independientemente.
- ▶ Se relaciona con muchas filas.

Ejemplo

Si muchos vehículos tienen la misma marca:

```
Marca(id_marca, nombre)
```

y no repetir “Toyota” en miles de filas.

Criterio clave

No se trata de crear muchas tablas porque sí, sino de reducir duplicación y representar bien las relaciones.

Llaves primarias: PK

Primary Key

La llave primaria identifica de manera única cada fila de una tabla.

Buenas candidatas

- ▶ patente de un vehículo.
- ▶ RUT de una persona
- ▶ Un id artificial autoincremental.

Malas candidatas

- ▶ Nombre.
- ▶ Color.
- ▶ Comuna.
- ▶ Cualquier dato que pueda repetirse.

Pregunta típica

“¿Cuántas llaves primarias necesito?” Una por tabla. Puede ser simple o compuesta. Para este último caso, por ejemplo: ID del curso + ID del estudiante en una tabla de notas

Llaves foráneas: FK

Foreign Key

Una llave foránea conecta una tabla con la llave primaria de otra tabla.



Lectura

Cada vehículo tiene una marca. La columna `id_marca` en **Vehiculo** apunta a `Marca(id_marca)`.

Cardinalidad: cómo leer relaciones

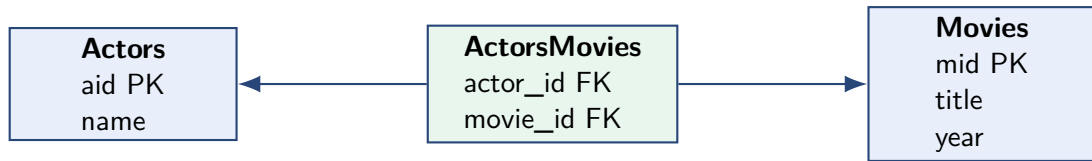
Tipo	Significado	Implementación típica
1 a 1	Una fila se relaciona con una sola fila.	FK con restricción UNIQUE.
1 a N	Una fila se relaciona con muchas.	FK en la tabla del lado N.
N a M	Muchas filas se relacionan con muchas.	Tabla intermedia.

Ejemplo N a M

Un actor puede aparecer en muchas películas. Una película puede tener muchos actores.

```
ActorPelicula(id_actor, id_pelicula)
```

Tabla intermedia: el patrón muchos-a-muchos



Buena práctica

La tabla intermedia debería tener llave primaria compuesta:

```
PRIMARY KEY(actor_id, movie_id)
```

Crear una tabla en SQLite

```
CREATE TABLE Marca (  
    id_marca INTEGER PRIMARY KEY,  
    nombre TEXT NOT NULL UNIQUE  
);  
  
CREATE TABLE Vehiculo (  
    vin TEXT PRIMARY KEY,  
    patente TEXT UNIQUE,  
    color TEXT,  
    id_marca INTEGER,  
    FOREIGN KEY (id_marca) REFERENCES Marca(id_marca)  
);
```

Regla general y buena práctica

La clave foránea (FK) indica explícitamente la columna referenciada en la otra tabla:

```
REFERENCES Marca(id_marca)
```

Aunque en SQLite puede omitirse cuando se referencia una clave primaria simple.

SQLite desde Python: conexión básica

```
import sqlite3

connection = sqlite3.connect("vehiculos.db") # si el archivo no existe, el
sistema lo creara automaticamente.
cursor = connection.cursor() # crea un "puntero" o herramienta que permite
ejecutar comandos SQL

cursor.execute("PRAGMA foreign_keys = ON") # Por defecto, SQLite no activa
esta verificacion para mantener la compatibilidad hacia atras con
versiones antiguas del programa.

# aqui ejecutamos comandos SQL con cursor.execute(...)
# al final:
connection.commit()
connection.close()
```

¿Dónde queda creada la base?

El archivo vehiculos.db queda en la misma carpeta donde se está ejecutando el notebook, salvo que indiquemos otra ruta.

Error típico: correr dos veces

Si ejecutamos dos veces:

```
CREATE TABLE Vehiculo (  
    vin TEXT PRIMARY KEY,  
    patente TEXT  
);
```

puede aparecer:

```
table Vehiculo already exists
```

Solución para notebooks

```
cursor.execute("DROP TABLE IF EXISTS Vehiculo")  
  
cursor.execute("""  
CREATE TABLE Vehiculo (  
    vin TEXT PRIMARY KEY,  
    patente TEXT  
)  
""")
```

Insertar datos de forma segura

```
marca = ("Toyota",)

cursor.execute(
    "INSERT INTO Marca(nombre) VALUES (?)",
    marca
)

connection.commit()
```

No hacer

```
cursor.execute("INSERT INTO Marca(nombre) VALUES ('%s')" % nombre)
```

Regla

Los valores externos entran con ?, no pegados al string SQL.

Insertar muchas filas

```
marcas = [  
    ("Toyota",),  
    ("Ford",),  
    ("Hyundai",)  
]  
  
cursor.executemany(  
    "INSERT INTO Marca(nombre) VALUES (?)",  
    marcas  
)  
  
connection.commit()
```

Cuándo usarlo

Cuando ya tenemos una lista de tuplas preparada desde un CSV, JSON o DataFrame. Además, ("Toyota ",) es para crear una tupla de un solo elemento.

Autoincremento y lastrowid

Problema común

No conviene asumir que el id generado será 1, luego 2, luego 3, especialmente si ya existían datos.

```
cursor.execute(  
    "INSERT INTO Marca(nombre) VALUES (?)",  
    ("Toyota",)  
)  
  
id_marca = cursor.lastrowid
```

Uso

Ese `id_marca` real se puede usar después como FK al insertar vehículos.

Cómo enfrentar un archivo CSV/JSON

1. Mirar columnas disponibles.
2. Identificar entidades candidatas.
3. Decidir llaves primarias.
4. Decidir relaciones y cardinalidades.
5. Crear tablas.
6. Insertar primero tablas “padre”.
7. Insertar después tablas que tienen FK.
8. Consultar para verificar.

Orden importante

No puedo insertar un vehículo con `id_marca = 3` si esa marca todavía no existe.

Verificar que las tablas existen

```
SELECT name  
FROM sqlite_master  
WHERE type = 'table';
```

```
PRAGMA table_info(Vehiculo);
```

Sirve para responder

- ▶ ¿Se creó la tabla?
- ▶ ¿Qué columnas tiene?
- ▶ ¿Qué tipos tienen las columnas?
- ▶ ¿Cuál es la llave primaria?

Consulta mínima: SELECT-FROM-WHERE

```
SELECT columna1, columna2  
FROM Tabla  
WHERE condicion;
```

Ejemplo

```
SELECT patente, color  
FROM Vehiculo  
WHERE color = 'Rojo';
```

Lectura

Desde la tabla Vehiculo, dame patente y color solo de los vehículos rojos.

JOIN: juntar tablas

```
SELECT V.patente, M.nombre
FROM Vehiculo V
JOIN Marca M
  ON V.id_marca = M.id_marca;
```

Lectura

- ▶ Vehiculo tiene la FK.
- ▶ Marca tiene la PK.
- ▶ El JOIN conecta ambas.

Error común

Olvidar la condición del JOIN. Eso puede generar combinaciones incorrectas entre todas las filas.

JOIN implícito vs JOIN explícito

Forma antigua

```
SELECT V.patente, M.nombre  
FROM Vehiculo V, Marca M  
WHERE V.id_marca = M.id_marca;
```

Forma recomendada

```
SELECT V.patente, M.nombre  
FROM Vehiculo V  
JOIN Marca M  
ON V.id_marca = M.id_marca;
```

Para estudiar

Ambas formas son equivalentes para un INNER JOIN, pero JOIN ... ON separa claramente las condiciones de unión de los filtros de la consulta.

Agregación: COUNT, AVG, MAX, MIN

```
SELECT id_marca, COUNT(*) AS cantidad  
FROM Vehiculo  
GROUP BY id_marca;
```

Lectura

Agrupamos vehículos por marca y contamos cuántos hay en cada grupo.

Diferencia clave

WHERE filtra filas antes de agrupar. HAVING filtra grupos después de agrupar.

GROUP BY + HAVING

```
SELECT M.nombre, COUNT(*) AS cantidad
FROM Vehiculo V
JOIN Marca M
  ON V.id_marca = M.id_marca
GROUP BY M.id_marca, M.nombre
HAVING COUNT(*) >= 3;
```

Qué responde

Marcas que tienen al menos 3 vehículos registrados.

Regla práctica

Si la condición depende de un agregado como COUNT, AVG, MAX, va en HAVING.

Subconsultas

```
SELECT patente, kilometraje
FROM Vehiculo
WHERE kilometraje > (
    SELECT AVG(kilometraje)
    FROM Vehiculo
);
```

Lectura

Primero se calcula el kilometraje promedio. Luego se seleccionan los vehículos que están sobre ese promedio.

EXISTS y NOT EXISTS

Idea

EXISTS pregunta si una subconsulta devuelve al menos una fila.

```
SELECT M.nombre
FROM Marca M
WHERE EXISTS (
    SELECT *
    FROM Vehiculo V
    WHERE V.id_marca = M.id_marca
);
```

Qué responde

Marcas que tienen al menos un vehículo asociado.

El patrón “todos cumplen”

Truco lógico

“Todos cumplen la condición” se puede escribir como:

No existe alguien que no la cumpla

```
SELECT M.nombre
FROM Marca M
WHERE NOT EXISTS (
    SELECT *
    FROM Vehiculo V
    WHERE V.id_marca = M.id_marca
    AND V.kilometraje > 100000
);
```

Lectura

Marcas para las cuales no existe un vehículo con más de 100.000 km.

Cómo opera

- ▶ SQLite toma una marca (por ejemplo, Toyota).
- ▶ Revisa la subconsulta para ver si Toyota tiene algún auto con más de 100,000 km.
- ▶ Si la subconsulta encuentra algo (un auto con 120,000 km), NOT EXISTS se vuelve Falso y esa marca queda descartada.
- ▶ Si la subconsulta se queda vacía (esa marca no tiene autos viejos), NOT EXISTS se vuelve Verdadero y la marca Toyota aparece en el resultado.

Errores frecuentes y cómo interpretarlos

Error	Qué suele significar
<code>table already exists</code>	Corriste dos veces la creación de tabla. Usa <code>DROP TABLE IF EXISTS</code> .
<code>no such table</code>	La tabla no fue creada o estás conectado a otra base.
<code>foreign key constraint failed</code>	Estás insertando una FK que no existe en la tabla padre.
<code>Cannot operate on a closed database</code>	Cerraste la conexión y luego intentaste usar el cursor.
<code>syntax error</code>	Falta coma, paréntesis, comillas o hay una palabra SQL mal escrita.

Plantilla de ejemplo

```
import sqlite3

db_path = "vehiculos.db"

connection = sqlite3.connect(db_path)
cursor = connection.cursor()
cursor.execute("PRAGMA foreign_keys = ON")

cursor.execute("DROP TABLE IF EXISTS Vehiculo")
cursor.execute("DROP TABLE IF EXISTS Marca")

cursor.execute("""
CREATE TABLE Marca (
    id_marca INTEGER PRIMARY KEY,
    nombre TEXT NOT NULL UNIQUE
)
""")
```

Plantilla de ejemplo

```
    cursor.execute("""
CREATE TABLE Vehiculo (
    vin TEXT PRIMARY KEY,
    patente TEXT UNIQUE,
    id_marca INTEGER,
    FOREIGN KEY (id_marca) REFERENCES Marca(id_marca)
)
""")

connection.commit()
connection.close()
```

Checklist para resolver ejercicios

1. ¿Qué archivos tengo y qué columnas traen?
2. ¿Cuáles son las entidades principales?
3. ¿Qué atributos pertenecen a cada entidad?
4. ¿Cuál es la PK de cada tabla?
5. ¿Dónde van las FK?
6. ¿Hay relaciones N a M que requieran tabla intermedia?
7. ¿En qué orden debo insertar los datos?
8. ¿Cómo verifico que la base quedó bien?
9. ¿Qué consulta SQL responde la pregunta?

Mini ejercicio

Supongamos que tenemos estos datos:

patente	marca	modelo	dueño
AA11	Toyota	Yaris	Ana
BB22	Toyota	Corolla	Luis
CC33	Hyundai	i20	Ana

Preguntas

- ▶ ¿Qué entidades aparecen?
- ▶ ¿Qué tabla tendría la FK?
- ▶ ¿Dónde evitaríamos repetir información?

Solución del mini ejercicio

Tablas posibles

- ▶ `Marca(id_marca, nombre)`
- ▶ `Modelo(id_modelo, nombre, id_marca)`
- ▶ `Dueno(id_dueno, nombre)`
- ▶ `Vehiculo(patente, id_modelo, id_dueno)`

Relaciones

- ▶ Una marca tiene muchos modelos.
- ▶ Un modelo puede aparecer en muchos vehículos.
- ▶ Un dueño puede tener muchos vehículos.
- ▶ Cada vehículo tiene una patente única.

Lo importante para avanzar eficientemente

- ▶ Primero modelar, después programar.
- ▶ Las PK identifican filas; las FK conectan tablas.
- ▶ Las tablas intermedias resuelven relaciones N a M.
- ▶ JOIN permite recuperar información distribuida.
- ▶ GROUP BY agrupa; HAVING filtra grupos.
- ▶ En SQLite, activar `PRAGMA foreign_keys = ON`.